

SAMCU: Secure and Anonymous Multi-Channel Updates in Payment Channel Networks

Jianhuan Wang¹, Shang Gao, *Member, IEEE*, Guyue Li², *Member, IEEE*,
Keke Gai³, *Senior Member, IEEE*, and Bin Xiao¹, *Fellow, IEEE*

Abstract—The Payment Channel Network (PCN) has emerged as an extensively adopted solution to address the scalability issues of Bitcoin by efficient off-chain updates. However, conflicts arise while existing update protocols are pursuing multiple goals of security, privacy, and expressiveness. In this work, we propose a new off-chain update protocol, Secure and Anonymous Multi-Channel Updates (SAMCU), which is developed on the basis of Unspent Transaction Output (UTXO). SAMCU aims at achieving goals of internal anonymity, balance security, and multi-channel updates simultaneously, which has not been done before. To achieve these goals, we exploit the technique of updating graph splitting (UGS) to make participants aware of only the identities of their neighboring sub-graphs, thereby ensuring internal anonymity in multi-channel updates. Then, to avoid security issues arising from equal sub-graphs, we further propose an Enable Payment Transaction Tree (EPTT) to guarantee balance security for each honest protocol participant. Moreover, we optimize the performance of our solution, reducing transaction fees by splitting transactions and the number of communication connections by hierarchical communication. To evaluate the performance of the SAMCU, we implement a prototype involving up to 100 updating payment channels. Experimental results demonstrate that SAMCU outperforms the state-of-the-art, resulting in approximately 70% savings in communication connections and a 66% reduction in on-chain transaction fees when the number of updating payment channels is 100.

Index Terms—Anonymity, multi-channel updates, payment channel networks (PCNs), Bitcoin, blockchain.

I. INTRODUCTION

THE advent of Bitcoin has revolutionized the concept of ledger systems by introducing a novel decentralized approach. With its inherent advantages (e.g., irreversibility,

pseudonymity, and transparency), Bitcoin has achieved significant development in the financial landscape. However, a critical drawback of this groundbreaking cryptocurrency is its limited throughput compared to traditional financial systems. This limitation has spurred the exploration of various avenues by community members and researchers to enhance Bitcoin's scalability.

To address the scalability challenge, numerous efforts have been directed towards off-chain protocols. Off-chain protocols reduce the burden on the blockchain by facilitating transactions off-chain between involved parties and resorting to on-chain interactions only when necessary. For instance, payment channels (PCs), which are among the most widely adopted off-chain protocols, require the publication of only two on-chain transactions on the blockchain. One transaction is dedicated to establishing the payment channel, while the other facilitates its closure. Further research has proposed the concept of *payment channel networks (PCNs)*, which enables two users without a direct payment channel to engage in off-chain transactions. By leveraging a multi-hop pathway comprising intermediate parties with existing payment channels, users can route payments through the network.

A crucial challenge that arises in PCN involving multiple payment channels is ensuring atomic updates. Atomicity refers to the property of a transaction being executed entirely or not at all, guaranteeing the consistency of the involved channels' states. Currently, there are two main approaches to achieve atomic updates across multiple payment channels: *Hash TimeLock Contract (HTLC)-based update protocols* and *Unspent Transaction Output (UTXO)-based update protocols*. HTLC-based update protocols [1] can achieve strong anonymity, and participants only need to generate transactions of constant size. However, it only supports multi-hop payments (MHPs, i.e., one-to-one transfers) and suffers from inefficient capital utilization caused by linear collateral requirements (i.e., the longer the MHP, the longer it takes for users' funds to be unlocked from HTLCs). In contrast, UTXO-based update protocols achieve multi-channel updates (MCUs, i.e., atomic updates across multiple channels with arbitrary topologies) and only require constant collateral, thereby enabling complex financial applications (e.g., crowdfunding and mass payment) on the Blockchain-like platforms. We illustrate examples of both MHPs and MCUs in Fig. 1.

Despite the advantages of UTXO-based update protocols [2], [3], [4], [5], they still have the following limitations:

Received 2 April 2024; revised 19 July 2024; accepted 13 August 2024. Date of publication 28 August 2024; date of current version 3 October 2024. This work was supported in part by HK RGC GRF under Project PolyU 15209822, in part by NSFC/RGC Joint Research Scheme under Project N_PolyU529/22 and Project 62261160391, and in part by the Natural Science Foundation on Frontier Leading Technology Basic Research Project of Jiangsu under Grant BK20222001. The associate editor coordinating the review of this article and approving it for publication was Prof. Guowen Xu. (*Corresponding author: Bin Xiao.*)

Jianhuan Wang, Shang Gao, and Bin Xiao are with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong (e-mail: 21122251r@connect.polyu.hk; shang-jason.gao@polyu.edu.hk; b.xiao@polyu.edu.hk).

Guyue Li is with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: guyuelee@seu.edu.cn).

Keke Gai is with the School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: gaikeke@bit.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIFS.2024.3451366>, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2024.3451366

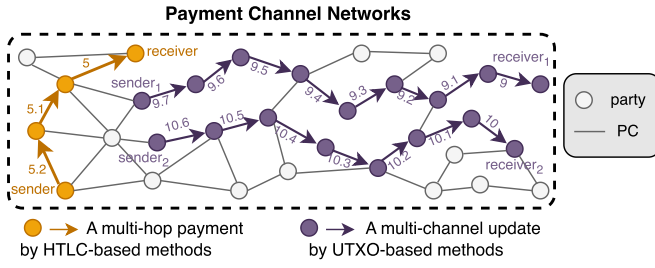


Fig. 1. Examples of the two update modes (i.e., Multi-hop payments and multi-channel updates) supported by HTLC-based and UTXO-based methods, respectively. The values on the payment channels represent the amounts transferred over the channels, with intermediaries within the payment network charging transaction fees (e.g., 0.1 coins) for supporting payments.

(a) **Lack of internal anonymity.** To guarantee the balance security of participants, UTXO-based methods require participants to be aware of the public addresses of all involved parties to generate the *Enable Payment* transactions collaboratively. This process results in the lack of *internal anonymity*, where intermediaries know the identities of payment senders and receivers. Existing UTXO-based methods [4] achieve internal anonymity only at the cost of sacrificing MCUs (i.e., only support MHPs), highlighting the significant challenge of enhancing internal anonymity in MCUs. (b) **Large size of transactions for on-chain enforcing payment.** In MCUs, each payment sender needs to find several parties as intermediaries to transfer the funds to her corresponding receiver. In some complex applications, such as multiple payment senders transferring funds to multiple payment receivers, the number of channels involved in the protocol can easily reach several dozen. Existing UTXO-based update protocols become impractical in complex applications because they require the enable payment transaction to have the same number of outputs as the number of channels involved. For instance, Thora [5] requires users to emit an enable payment transaction of size 3.9Kb (transaction fees of approximately 7.7 USD in Dec. 2023) to the blockchain when their channels are disputed and the number of updating payment channels reaches 100. (c) **Quadratic number of communication connections.** All parties participating in the UTXO-based update protocols need to communicate with each other, necessitating the creation of a quadratic number of connections. This communication overhead becomes substantial as the number of channels increases. The large number of connections imposes limitations on the simultaneous participation of users in the MCUs, as the number of connections that personal computers can establish simultaneously is limited.

To address these limitations, we introduce a **Secure and Anonymous Multi-Channel Updates (SAMCU)** for PCN that achieves internal anonymity and balance security for MCUs. To achieve PCN internal anonymity, we firstly introduce **Updating Graph Splitting (UGS)**, which divides an updating graph (UG) into several chain-like sub-graphs (SGs), where an *updating graph (UG)* represents the topology of multiple channels needing updates. Based on UGS, our protocol ensures that each party knows only the identities of the parties within the SG where she belongs, rather than all the parties in the UG, thereby preserving anonymity for both senders and receivers.

TABLE I
STATE OF THE ART IN PCNS

	Update Mode	Collateral	Identity Privacy	Balance Security	On-chain TX size
LN [1]	MHP	$O(n)$	Yes	Yes	$O(1)$
AMCU [2]	MCU	$O(1)$	No	No	$O(n)$
PT [3]	MHP	$O(\log_n)$	No	Yes	$O(1)$
Blitz [4]	MHP	$O(1)$	Yes	Yes	$O(1)$
Thora [5]	MCU	$O(1)$	Internal leak	Yes	$O(n)$
Our solution	MCU	$O(1)$	Yes	Yes	$O(\frac{n}{m} + m)$

We also propose a novel enable payment transaction tree with necessary timelocks and signature locks to ensure balance security for every honest party while maintaining internal anonymity.

Secondly, our method utilizing UGS reduces the size of enable payment transactions from $O(n)$ to $O(\frac{n}{m} + m)$, where n denotes the number of updating channels, m denotes the number of sub-graphs, as illustrated in Table I. Because participants only need to generate transactions containing the addresses of parties within their sub-graphs, excluding the need for addresses of all parties in the updating graph. Thirdly, to reduce the number of communication connections, we propose a hierarchical communication strategy based on UGS where each sub-graph designates a subset of parties as key parties. These key parties establish complete connections among themselves to facilitate inter-sub-graph information exchange. Parties in different SGs can communicate via these key parties without establishing direct connections, thereby reducing cross-sub-graph connections.

Our contributions can be summarized as follows:

- **SAMCU Protocol.** We propose SAMCU, the first UTXO-based update protocol to achieve multi-channel updates, anonymity, and constant collateral simultaneously in Section III and Section IV. Compared with previous protocols for MCUs, SAMCU achieves (i) internal anonymity (i.e., sender/receiver privacy), (ii) smaller sizes & lower fees of on-chain enforcing payment transactions, and (iii) fewer communication connections.
- **Security and Privacy Analysis:** We conduct a security and privacy analysis of SAMCU in Section V. We show that each honest party engaging in the protocol can ensure her *balance security* by following the protocol's procedures. In addition, we prove that SAMCU achieves *internal anonymity* when the protocol runs without disputes.
- **Implementation and Evaluation:** We implement a prototype of SAMCU protocol.¹ In addition, we perform a comprehensive performance evaluation of SAMCU protocol in Section VI. Experimental results show that SAMCU effectively reduces transaction size and number of communication connections compared to the state-of-the-art, especially in large-scale multi-channel updates (involving 100 updating PCs).

¹<https://github.com/SAMCU-PCN/SAMCU/>

II. BACKGROUND

A. Unspent Transaction Output (UTXO) Transactions

The UTXO model serves as the foundational framework for representing funds in many UTXO-based blockchains (e.g., Bitcoin). In contrast to account-based blockchains that employ a key-value database in the form of $\{\text{account} : \text{balance}\}$ to store user assets, UTXO-based cryptocurrencies utilize a series of UTXOs to represent user assets. The total balance of a user is the sum of the balances of all UTXOs they own. In the UTXO model, a UTXO can be represented as a tuple: $\theta = (\text{TXID}, \text{index}, \text{value}, \text{ls})$. TXID is the ID of the transaction that created θ , *index* is the position of the output within that transaction, *value* is the amount of balance locked in θ , and *ls* is the conditions necessary for spending the balance in θ .

In this paper, we utilize two widely adopted signature conditions and two widely adopted timelock conditions in Bitcoin-like blockchains. The condition that requires a user's signature is represented by $\text{OneSig}(u)$. Likewise, the condition that requires the signatures of multiple users is represented by $\text{MultiSig}(u_1, \dots, u_n)$. We use $\text{AbsT}(T)$ to represent the absolute timelock condition, indicating that the output with this condition should be executed only before time T . We use $\text{RelT}(\Delta)$ to represent the relative timelock condition, meaning the output with this condition can only be unlocked Δ time after it is posted to the blockchain. Due to the fact that transaction fees are determined by the size of transactions, directly storing complex locking scripts like MultiSig in UTXOs leads to higher transaction fees on the blockchain. Moreover, storing locking scripts in UTXOs on the blockchain compromises privacy, as external observers can easily access the script's content in UTXOs. To mitigate these concerns, BIP 16 [6] introduced the standardization of Pay-to-Script-Hash (P2SH) transactions. In a P2SH transaction, a UTXO only stores $\mathcal{H}(\text{ls})$, which is a hash value of the locking script *ls*. Users then need both the unlocking script and locking script to redeem funds from the UTXO.

B. Payment Channels (PCs)

Payment channels are a popular solution for enhancing blockchain scalability, allowing only for off-chain payments between two parties. The Lightning Network (LN) [1] is a significant implementation of this technology, with the total value locked on the network reaching 141 million USD as of June 2023. The key idea behind this technique is to enable two parties to make multiple payments using a payment channel while only recording two transactions on the ledger. The operations of a payment channel include opening, updating, and closing. To open a channel, both parties lock a certain amount of funds in a 2-of-2 multi-signature output. This output requires signatures from both parties to redeem their funds, ensuring their balance security. During the off-chain payments, they can create new transactions to update the channel's state (i.e., the balance of the two parties in the channel). These updates occur off-chain, enabling efficient multiple payments within the channel without sending transactions to the blockchain. When these two parties decide to close

the channel, one of them can submit the final negotiated transaction to the blockchain containing signatures from both parties. Once this transaction is included by the blockchain, the parties can redeem their funds from the 2-of-2 multi-signature output and conclude the use of the channel.

C. Payment Channel Networks (PCNs)

PCNs (e.g., LN [1] and Eclair [7]) are an extension of PCs that establish a network of interconnected channels that aim at enhancing the efficiency of PCs. When two users who do not have an established PC wish to conduct off-chain transactions, they can leverage an existing network path that connects them to facilitate their transfer. There are primarily two update modes in PCN: *Multi-hop payments (MHPs)* [1], [3], [4], [8], [9] and *Multi-channel updates* [2], [5]. One critical issue in PCNs is the atomicity problem, which refers to the guarantee that the states of all channels in payment are either all updated together or none are updated, ensuring that users do not suffer losses due to partial execution. Currently, there are two kinds of approaches to achieve atomicity in PCNs: *HTLC-based update protocols* [1] and *UTXO-based update protocols* [2], [3], [4], [5]. We introduce these methods below and show the comparison of existing approaches in Table I.

1) *HTLC-Based Update Protocols*: HTLC-based update protocols, as proposed by the Lightning Network (LN) [1], ensure atomicity across a multi-hop payment, which can be modeled as $\text{sender} \rightarrow v_1, \dots, v_n \rightarrow \text{receiver}$. An HTLC can be constructed by two parties sharing an open payment channel (e.g., Alice and Bob) and allows Alice to lock α coins that can be released only if the contract's conditions (i.e., a hash lock and a time lock) are fulfilled. The core idea for ensuring atomicity is that each pair of adjacent parties in the multi-hop path creates HTLCs bound by the same hash lock. The preimage of this hash is known only to the receiver. When the receiver uses the preimage to redeem her funds, parties from v_n to v_1 sequentially learn the secret and use it to unlock the HTLCs they created with the previous party. This mechanism ensures that either all HTLCs are unlocked or none are, maintaining the atomicity of the MHP. Further studies employ signature algorithms to replace HTLC to enhance the security to defense against *the wormhole attacks*, and interoperability of MHPs [8], [9]. However, existing HTLC-based update protocols and signature-based update protocols still have the following limitations: (a) *Low Expressiveness*. These protocols for MHPs impose limitations on the complexity of payment scenarios, as it does not support complex structures (i.e., crowdfunding) (b) *Liner Collateral*. The duration of the locking phase for the payment sender increases linearly with the increasing number of intermediaries involved, as each intermediary needs to redeem their funds sequentially.

2) *UTXO-Based Update Protocols*: Atomic Multi-Channel Update (AMCU) protocol [2] is the first attempt to solve the long-standing collateral challenge [10], aiming to achieve *constant collateral payments*. It utilizes the UTXO mechanism, whereby the amount of a transaction is atomically transferred from one or more inputs to one or more outputs to achieve atomicity across multiple channels. This protocol also supports

MCUs, enabling users to perform more complex applications on UTXO-based blockchains, such as crowdfunding, mass payments, and PCN rebalancing. However, this protocol sacrifices the identity privacy of payment senders/receivers. Jourenko et al. [3] find that AMCU is not secure. It suffers from *channel closure attacks* [3], where malicious intermediaries collude to perform a double-spending attack to the enable payment transaction, rendering it unaccepted by the blockchain and compromising the atomicity of the protocol. In response to this issue, they proposed the payment trees (PT) protocol that utilizes the UTXO mechanism to ensure the atomicity of MHPs. However, it comes with a higher collateral requirement compared to AMCU, increasing from $\mathcal{O}(1)$ to $\mathcal{O}(\log_n)$. Furthermore, this protocol inherits the privacy limitations of AMCU. Aumayr et al. [4] introduced Blitz, which enables MHPs in a single round of communication, contrasting with LN's two-phase commit approach. This protocol provides defense against wormhole attacks while maintaining constant collateral. Additionally, the Blitz protocol achieves identity privacy for MHPs' sender and receiver. Aumayr et al. [5] propose an alternative approach, called Thora, to mitigate the *channel closure attack* by having each channel receiver create her own enable payment transaction. This UTXO-based transaction requires the receiver's signature to become valid, thereby defending against malicious users attempting a channel closure attack. However, existing UTXO-based update protocols still fail to simultaneously achieve balance security, internal anonymity, and multi-channel updates.

III. SOLUTION OVERVIEW

A. System Model and Threat Model

1) *System Model*: A PCN can be modeled as $\mathcal{G} := (\mathcal{V}, \mathcal{E})$. The set of the nodes $\mathcal{V}$ represents the users' addresses in the blockchain. The set of weighted edges $\mathcal{E}$ represents the opened payment channels in the PCN. Every weighted edge $(X_{v_1}, X_{v_2}) \in \mathcal{E}$ represents the balances of $v_1, v_2 \in \mathcal{V}$ stored in the PC. In this paper, we focus on MCUs in PCN. In an MCU, one or more payment senders transfer funds to one or more payment receivers. We refer to parties that only spend funds as payment senders, and those parties only receive funds as payment receivers. By utilizing specific routing algorithms [11], [12], [13], [14], these parties can discover an updating graph $\text{UG} := (\{\gamma_i, \alpha_i\}_{i=1}^n)$ to execute the off-chain payments, where $\{\gamma_i\}_{i=1}^n \subset \mathcal{G}$ represents the set of PCs linking all payment senders and receivers and $\{\alpha_i\}_{i=1}^n$ represents the transfer amounts for these PCs. Apart from payment senders and receivers, parties involved in the MCUs are referred to as intermediaries whose fund changes are limited to the payment fees they charge.

In our protocol, we assume the existence of a secure and authenticated communication channel among protocol participants, like TLS channel. We also assume that honest parties remain online throughout the protocol, like [4].

2) *Threat Model*: In this paper, we consider the following two types of attacks in our model: (1) Collusion Attacks (e.g., the channel closure attacks), some malicious parties may conspire to disrupt the atomicity of updates to compromise the

balance security of honest parties and steal funds from honest parties; and (2) Identity Privacy Attacks, we consider a party's identity privacy compromised when their blockchain address is revealed. We assume that payment senders and payment receivers know each other's identity information, which is a reasonable assumption as payments cannot occur without knowing addresses. However, intermediate parties who assist in the payment are curious about the identities of payment senders and receivers and may try to infer this information.

B. Security and Privacy Goals

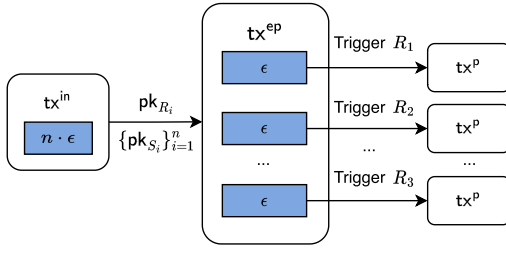
1) *Balance Security* [1], [4]: Independent of whether the protocol runs without disputes, no honest intermediate party within the update protocol loses her funds. Note that we do not consider the balance security for malicious parties, as they can compromise the atomicity of the channels they are involved in at the cost of their own funds. However, a rational attacker would not do so.

2) *Internal Anonymity (Sender/Receiver Privacy)* [4]: When the protocol runs without disputes (i.e., no party forcibly closes her payment channel on-chain), if an adversary controlling an intermediary can not determine whether any of his neighbors is one of the payment senders or one of the payment receivers, then we regard that the protocol achieves internal anonymity. This notion of anonymity is more stringent compared to that of existing MCUs protocols [5], as these protocols' preservation of parties' identity privacy is limited to external adversaries outside the protocol, while internal parties still know their identities.

C. Our Solution

We present a novel UTXO-based update protocol that simultaneously achieves MCUs, internal anonymity, and balance security. We describe our protocol in an incremental way. We first recall the method of employing *Trigger and Response* pattern, a strategy widely utilized in previous UTXO-based update protocols, which solely achieve MCUs and balance security. Then, we give a naive approach utilizing Updating Graph Splitting (UGS) to achieve MCUs and internal anonymity while sacrificing balance security. Finally, we introduce the Enable Payment Transaction Tree (EPTT) to achieve balance security for the naive approach.

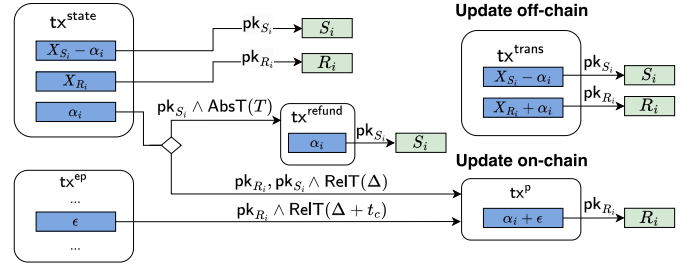
1) *Trigger and Response Pattern*: The current UTXO-based update protocols mainly employ the *Trigger and Response* pattern to address the challenge of atomic updates across multiple channels. Specifically, atomic updates require that given a $\text{UG} := \{\{\gamma_i, \alpha_i\}_{i=1}^n\}$, the expected state of the channels by the timeout T has only the following clauses: (a) either all channel states are successfully updated off-chain (or enforcing payment on-chain), allowing all the channel receivers of $\{\gamma_i\}_{i=1}^n$ to concurrently receive the payment value $\{\alpha_i\}_{i=1}^n$, respectively; or (b) all the receivers in $\{\gamma_i\}_{i=1}^n$ does not receive $\{\alpha_i\}_{i=1}^n$, respectively. To solve this challenge, protocols employing the *Trigger and Response* pattern are divided into two primary operations: (1) The *Trigger* operation, where if any channel receiver forcibly claims her coins from her channel before T , she must publish an *Enable Payment* transaction to the

Fig. 2. Overview of the transactions for *Trigger* operation.

blockchain, signaling a global event to notify other channel receivers. (2) The *Response* operation, where other channel receivers, upon observing the presence of the *Enable Payment* transaction on the blockchain, will react by forcibly claiming their coins from their channels. These operations can be executed in parallel, thereby ensuring constant collateral. Ultimately, if no channel updates occur by the time T expires, all channel states will revert to their status prior to the update. This pattern, in contrast to the HTLC-based update protocol, has the advantage of being applicable to updating graphs of any topological structure and constant collateral. We give an intuitive explanation of the *Trigger* operation and *Response* operation below.

a) *Transactions for trigger operation*: Some existing works (e.g., Blitz) rely on a trusted user to perform the trigger operation. Contrastingly, some existing works (e.g., Thora) eliminate the need for a trusted user by allowing any channel receiver to act as a trigger. In our approach, we adopt the latter strategy, where every channel receiver, denoted by R_i , needs to generate transactions for the trigger operation: tx^{in} and tx^{ep} , as shown in Fig. 2. tx^{in} is used to allocate coins for R_i , where the receiver needs to lock the required funds $n \cdot \epsilon$ in one of the outputs of tx^{in} , with ϵ representing the minimum value supported by the blockchain (e.g., 1 satoshi in Bitcoin). tx^{ep} is a crucial transaction that triggers *Response* operations for all receivers. Its input is one of the outputs of tx^{in} , and it generates an output for each channel receiver R_i , serving as a prerequisite for R_i to execute the *Response* operation. Each output has a value of ϵ and a locking script of $OneSig(R_i)$. Once the transaction tx^{ep} is generated, it is sent to each channel sender to enable her to generate the corresponding update contract for the *Response* operation based on the received tx^{ep} .

b) *Update contract for response operation*: Upon receiving a set of transactions $\{tx^{ep}\}$ from all channel receivers, each channel sender collaborates with her respective channel receiver to generate an update contract. Fig. 3 shows an example of an update contract for the i -th channel γ_i , where we assume tx^{ep} as the trigger transaction received from another channel receiver. The balance states of S_i and R_i in the channel are represented as X_{S_i} and X_{R_i} , respectively. S_i first uses tx^{state} to update the state of γ_i , which can be seen as a transitional state, rendering her channel *ready* for the transfer of α_i . In tx^{state} , the sender's balance is split into two outputs: one output with a value of $X_{S_i} - \alpha_i$ and another output with a value of α_i . For the second output, its condition is specified as $(MultiSig(R_i, S_i) \wedge RelT(\Delta)) \vee (OneSig(S_i) \wedge AbsT(T))$.

Fig. 3. Overview of the update contract for *Response* operation.

The timelock condition $AbsT(T)$ for the input of tx^{refund} ensures that the channel sender can only refund the payment value α_i after T . Meanwhile, the timelock condition $RelT(\Delta)$ for the input of tx^p guarantees that the execution priority of tx^{refund} is higher than tx^p after T , thereby eliminating the possibility that the channel receiver redeem α_i by tx^p after T . Then, the state of γ_i has the following clauses: (1) S_i can wait for the expiration of the timeout T and then uses tx^{refund} to redeem α_i . This can be viewed as rolling back γ_i to the state before the update; or (2) When R_i observes one tx^{ep} (may be posted by her own) being confirmed on the blockchain, she can publish the corresponding tx^p to the blockchain to enforce her payment before T . This can be seen as forcibly updating the channel state to the state of successfully sending α_i to R_i in an on-chain way; or (3) S_i and R_i honestly update the state of γ_i using a new off-chain state transaction tx^{trans} . This transaction contains two outputs, representing the states of the S_i and R_i after transferring α_i .

When the update contract for a payment channel is accurately created, the channel receiver will send a notification message to all channel senders. Upon receiving all notification messages from all channel receivers, a channel sender sends her signature for the corresponding tx^{ep} to each channel receiver. This process ensures that once a tx^{ep} becomes valid (having obtained all necessary signatures), all transactions and update contracts are generated, and each channel γ_i becomes *ready* for the transfer of α_i . Then, if any channel receiver needs to enforce her payment on-chain when there is a dispute on her channel, she will post tx^{in} , tx^{ep} , and tx^p (response to her own trigger) to the blockchain. Upon observing tx^{ep} being confirmed in the blockchain, other channel receivers will close their channels and publish their corresponding tx^p to the blockchain to conduct the *Response* operations, ensuring that all channel receivers can enforce their corresponding payment on-chain simultaneously. The timelock condition $RelT(t_c + \Delta)$ for θ_{tx^{ep}/tx^p} ensures sufficient time for other channel receivers to close their channels and subsequently publish their tx^p transactions to the blockchain, where θ_{tx_1/tx_2} represents the output connecting tx_1 and tx_2 , and t_c represents the maximum time for closing a payment channel.

However, when engaging in protocols that follow the *Trigger* and *Response* pattern, intermediaries acting as channel receivers need to know the list of addresses of all channel receivers (denoted by $\mathbb{R}$) to construct tx^{ep} . In addition, they also need to know the list of addresses of all channel senders (denoted by $\mathbb{S}$) to unlock the input of their corresponding

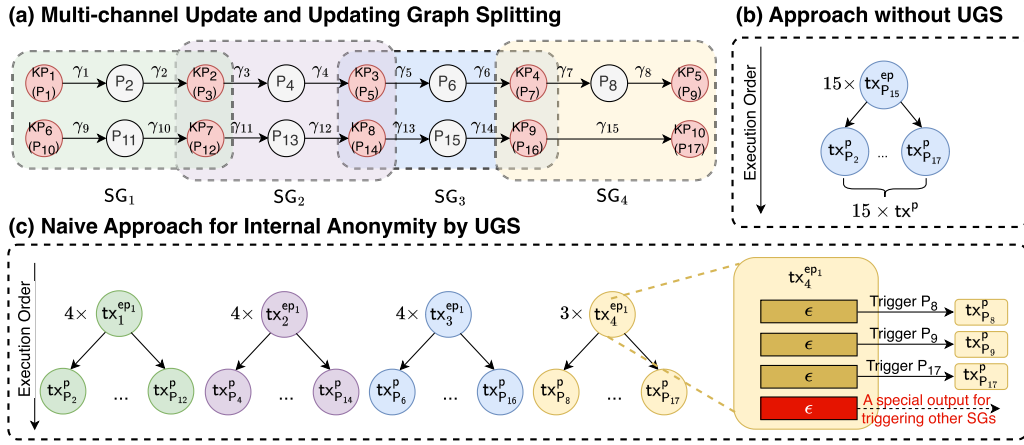


Fig. 4. (a) An example of updating graph splitting. The updating graph depicts a two-to-two payment. The two payment senders transfer their funds to the two payment receivers through 7 and 6 intermediaries, respectively. This updating graph is divided into four sub-graphs by UGS, denoted as SG₁, ..., SG₄. Each sub-graph has 3 or 4 channels. (b) The traditional approach employs the *Trigger and Response* pattern, where a channel receiver initiates an enable payment transaction with 15 outputs to trigger 15 channel receivers, respectively. (c) The naive approach utilizes UGS to achieve internal anonymity, where each channel receiver generates a tx^{ep}_{P₁} to trigger other channel receivers. tx^{ep}_{P₁} consists solely of outputs that trigger each receiver within her SG, as well as a special output containing global information for triggering channel receivers in other SGs. Consequently, a channel receiver only knows the identities within her own SG, rather than the identities of all channel receivers, to create tx^{ep}_{P₁}.

tx^{ep}. Consequently, this information allows participants to easily determine that $\mathbb{S} \setminus \mathbb{R}$ represents the addresses of payment senders and $\mathbb{R} \setminus \mathbb{S}$ represents the addresses of payment receivers.

2) *Achieving Internal Anonymity by Updating Graph Splitting:* We introduce a naive approach to ensure internal anonymity for UTXO-based update protocols through our novel technique, termed updating graph splitting. We present an example (in Fig. 4) to illustrate the process of UGS and show how UGS can be utilized to achieve internal anonymity.

Given a UG $\text{UG} := (\{(\gamma_i, \alpha_i)\}_{i=1}^n)$, we split it into multiple SGs to ensure anonymity for the payment senders and receivers. We use an example to illustrate the structure of a split graph shown in Fig. 4(a), where the graph UG is divided into four SGs, denoted by SG₁, ..., SG₄. Note that the splitting involves dividing the channels into several subsets, leading to overlapping parties across the SGs. For example, P₃ and P₁₂ belong to both SG₁ and SG₂. These overlapping parties, along with the payment senders/receivers, collectively form the key parties (KPs), denoted as KP_i. We define parties other than the key parties as normal parties (NPs). To better introduce our protocol, we introduce additional notions for these channels and parties. For each SG SG_i, the individual channels in SG_i are labeled as $\{\gamma_{i,j}\}_{j=1}^{c_i}$, where c_i represents the number of channels within SG_i. The j -th parties in SG_i are represented by P_{i,j}, and $\mathbb{P}_i$ represents the set $\{P_{i,j}\}_{j=1}^{n_i}$, where n_i represents the number of parties within SG_i. Similarly, S_{i,j} and R_{i,j} represents the sender and receiver of the j -th channel in SG_i, respectively. $\mathbb{S}_i$ and $\mathbb{R}_i$ represent the set $\{S_{i,j}\}_{j=1}^{c_i}$ and $\{R_{i,j}\}_{j=1}^{c_i}$, respectively.

a) *Principle for UGS:* To better introduce the principle, we first define the concept of a *chain*, which refers to a series of PCs that link a payment sender to a payment receiver. A UG must consist of several chains. For example, the UG in Fig. 4(a) consists of two chains (i.e., P₁ to P₉ and P₁₀ to P₁₇). The splitting must adhere to the following principles: (a) Payment senders and receivers must be key parties. They

need to know more information to ensure their balance security than normal parties; (b) Each SG must contain at least one PC from each chain. This ensures that the number of segments in each chain corresponds to the number of SGs, and (c) The UG must be divided into at least three SGs. Currently, the average number of hops required for a one-to-one transfer between any two users in PCN is 9 (as of Feb. 2024 in LN) [15]. Therefore, it is not difficult to split an updating graph into three or more SGs. For small updating graphs that cannot be split into more than three SGs, we discuss how our protocol ensures their internal anonymity in Appendix A. We give an example algorithm for UGS in Appendix B.

Upon UGS, we establish a privacy requirement critical for our protocol to achieve internal anonymity within MCUs. Specifically, this privacy requirement requires that each party in UG knows only the public addresses of parties within their respective SGs, rather than all parties' public addresses in UG, thereby ensuring internal anonymity. For example, P₄, who belongs to SG₂, knows only the public addresses of the parties in SG₂ (i.e., P₃ to P₅ and P₁₂ to P₁₄). KP₃, who belongs to both SG₂ and SG₃, only knows the public addresses of the parties in both SG₂ and SG₃ (i.e., P₃ to P₇ and P₁₂ to P₁₆). Therefore, an intermediary can not know the public addresses of payment senders and receivers simultaneously when the number of SGs exceeds two. Even if a party and one of the payment senders/receivers are in the same SG (e.g., P₇ and P₉), the intermediary P₇ cannot determine if the known neighbors (e.g., P₉) are payment senders/receivers or just other intermediaries.

b) *Naive approach:* We propose a naive approach to fulfill the privacy requirements for achieving internal anonymity in MCUs. After splitting the UG, we require parties within each SG to generate transactions (i.e., tx^{ep} and tx^p) solely with her SG's parties for their respective SGs. For the generation of transactions tx^{ep} and tx^p, parties are only required to know the identities of parties in their respective SGs. However, tx^{ep} in Fig. 2 can solely trigger parties within the same SG and

cannot trigger parties of other SGs. To overcome this issue, we introduce a novel enable payment transaction, denoted as tx^{ep_1} , as depicted in Fig. 4(c). Unlike tx^{ep} , this transaction has an extra output that serves as a notifier, alerting channel receivers in other SGs to respond. The value of this output is ϵ , and the locking script of the output is set to the notifier of this specific MCU (e.g., the sid of this update). Consequently, when any tx^{ep_1} is published on the blockchain, all channel receivers in other SGs can respond to this trigger by posting tx^{ep_1} for their SGs and her own tx^{p} upon detecting a UTXO labeled with the notifier of this MCU.

However, this naive approach is vulnerable to *griefing attacks*, compromising its balance security. With the timelock setting of $\theta_{\text{tx}^{\text{ep}_1}/\text{tx}^{\text{p}}}$, a rational trigger should post her tx^{ep_1} to the blockchain before $T - 3\Delta - t_c$, which ensures that tx^{ep_1} will be on the blockchain before $T - 2\Delta - t_c$. Then, after waiting for $\Delta + t_c$, when the output of tx^{ep_1} is unlocked, the trigger can post tx^{p} to the blockchain before $T - \Delta$, and tx^{p} will be confirmed before T . If a malicious trigger delays posting her tx^{ep_1} to the blockchain until $T - 3\Delta - t_c$, although the channel receivers in the trigger's SG have enough time to post their tx^{p} before $T - \Delta$ (accepted before T), other channel receivers do not have enough time to post tx^{ep_1} for enabling payment for their SGs and tx^{p} to the blockchain, thereby breaking atomicity among all SGs.

3) *Achieving Balance Security by EPTT*: To ensure balance security, we introduce the enable payment transaction trees that follow the *Trigger and Response* pattern. Specifically, we require that each channel receiver creates a transaction tree where they act as the trigger. Fig. 5 shows an example of an EPTT, corresponding to the updating graph depicted in Fig. 4. In this tree, $R_{3,3}$ is the trigger, with the other channel receivers acting as responders. This tree is structured into five layers. The first and second layers are tx^{in} and tx^{ep_1} , respectively, facilitating the enforcing payment for the trigger's SG, SG_3 . The third layer includes payment transactions from channel receivers in SG_3 and a special transaction tx^{relay} , which initiates response operations for channel receivers in other SGs. tx^{relay} outputs for each SG, enabling tx^{ep_2} of all other SGs, respectively. Notably, tx^{ep_2} is similar to tx^{ep} , with the primary difference being that its input comes from tx^{relay} instead of tx^{in} . The fifth layer consists of payment transactions from channel receivers in other SGs responding to the trigger. For simplicity in describing our protocol, we use tx^{p_1} and tx^{p_2} instead of tx^{ep_1} and tx^{ep_2} , respectively.

Compared to the naive approach, the structure of EPTT prescribes a sequential execution order for enable payment transactions of SGs: initially triggering tx^{ep_1} for the parties in the SG where the trigger is in, followed by tx^{ep_2} for parties in other SGs. This allows us to employ timelock settings to safeguard against griefing attacks. We describe the setting of timelocks of the transaction in EPTT, as shown in Fig. 5. We retain the timelock condition for $\theta_{\text{tx}^{\text{ep}_2}/\text{tx}^{\text{p}_2}}$ as $\text{RelT}(\Delta + t_c)$ and do not set any timelock conditions on $\theta_{\text{tx}^{\text{ep}_1}/\text{tx}^{\text{p}_1}}$ and $\theta_{\text{tx}^{\text{relay}}/\text{tx}^{\text{p}_2}}$. In these timelock settings, a rational trigger must post tx^{ep_2} before $T - 3\Delta - t_c$ for her tx^{p_2} to be confirmed by the blockchain before T . Consider the blockchain delay

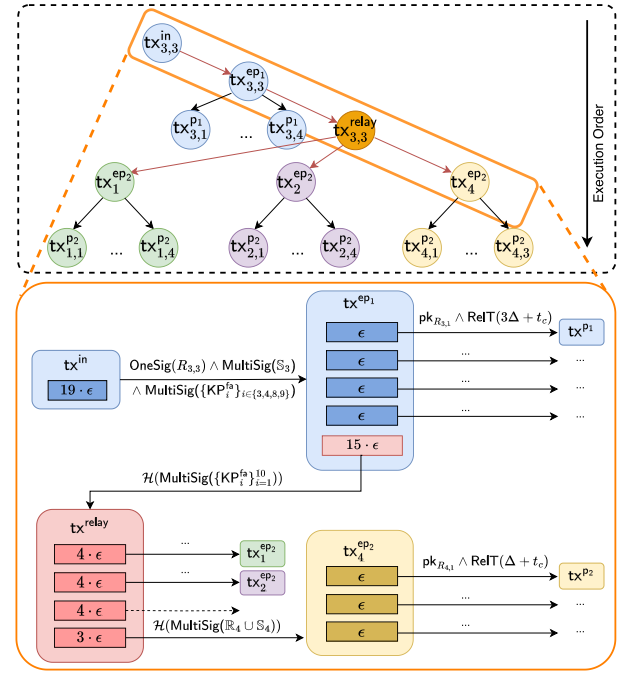


Fig. 5. An example of an EPTT that adopts the *Trigger-Response* pattern. In this tree, $R_{3,3}$ acts as the trigger, being the first channel receiver to publish tx^{in} and tx^{ep_1} for enforcing payment. Upon observing tx^{ep_1} on the blockchain, other receivers can respond to this trigger, i.e., they publish their corresponding transactions following the tree to enforce their payments. Note that tx^{relay} includes four outputs (1 output per SG). Since SG_3 does not require additional triggering by tx^{ep_2} , the output for SG_3 remains unspent. For clarity, it is hidden from the illustration.

Δ , tx^{relay} and tx^{ep_1} must be posted before $T - 4\Delta - t_c$ and $T - 5\Delta - t_c$, respectively, to allow responders to post tx^{ep_2} to the blockchain before $T - 3\Delta - t_c$. To ensure that the trigger will definitely post tx^{ep_1} before $T - 5\Delta - t_c$, we set $\text{RelT}(3\Delta + t_c)$ for $\theta_{\text{tx}^{\text{ep}_1}/\text{tx}^{\text{p}_1}}$. With this setting, a rational receiver must post tx^{ep_1} before $T - 5\Delta - t_c$ for her tx^{p_1} to be confirmed by the blockchain before T .

We require parties from different SGs to collaborate in creating their transaction trees, with key parties acting as bridges for information dissemination between the SGs. To preserve key parties' anonymity, we require key parties to generate one-time-use fresh addresses $\{\text{KP}_i^{\text{fa}}\}_{i=1}^n$ and use them as their identifiers for communication among themselves without revealing their real identities. We describe the creation of the $R_{3,3}$'s tree as an example (cf. Fig. 5). After $R_{3,3}$ creates a transaction tx^{ep_1} containing SG party identity information through communication with her SG's parties, she sends tx^{ep_1} to the key parties in SG_3 . Then, the key parties generate the corresponding tx^{relay} and send it to key parties in other SGs. These key parties in other SGs then collaborate with parties from their SGs to create the final two layers of the tree (each SG's tx^{ep_2} and the corresponding tx^{p_2}), respectively. Information transfer between SGs consists solely of details regarding tx^{relay} transactions conducted by key parties. As tx^{relay} 's input's unlocking script is $\text{MultiSig}(\{\text{KP}_i^{\text{fa}}\}_{i=1}^n)$, and its output for each SG is a hash of a locking script, no identity information is exposed. Furthermore, the generation of transaction trees for every channel receiver can be executed in parallel, thereby not increasing the time complexity.

There are two ways in which the atomicity of an EPTT can be compromised: (1) If the EPTT becomes valid (i.e., all transactions in the tree are signed) when only a portion of PCs are *ready*, then the receivers in payment channels not yet ready will be unable to enforce payment using $\text{tx}^{\text{p1/p2}}$ in case of disputes in their channels. (2) The outputs of transactions are double-spent maliciously by unexpected transactions.

a) *Ensure readiness of all PCs*: To prevent the transaction tree from becoming valid before all channels update their channel states by tx^{state} , we add a signature condition $\text{MultiSig}(S_i \cup \{KP_k^{\text{fa}}\}_{\forall KP_k \in SG_i})$ to tx^{ep1} , add a signature condition $\text{MultiSig}(\{KP_i^{\text{fa}}\})$ to tx^{relay} , and add a signature condition $\text{MultiSig}(S_i)$ to tx^{ep2} , as shown in Fig. 5. After a channel sender successfully updates her channel by tx^{state} , she will sign $\{\text{tx}^{\text{ep1/ep2}}\}$ for all channel receivers' trees in her SG and then sends these signatures to the parties in her SG. After receiving these signatures from all channel senders in her SGs, the key party will inform other key parties that the SGs she is responsible for are all *ready* by sending her signatures of $\{\text{tx}^{\text{relay}}\}$. The key party only sends her signatures of $\{\text{tx}^{\text{ep1}}\}$ to the channel receivers in her SGs after confirming that all other payment channels have been ready. This process ensures that when a channel receiver receives all the signatures to satisfy the signature conditions of tx^{ep1} , every payment channel in UG has been ready.

b) *Prevent double spending*: To prevent double spending, we carefully design the signature conditions of locking scripts for transactions. We add a signature condition to the input of tx^{ep1} , requiring the signature of the owner of tx^{ep1} to unlock it. As a result, for the first receiver $R_{i,j}$ executing the trigger and response operations, all inputs of tx^{in} , tx^{ep1} , and tx^{p1} require her signature to unlock the corresponding outputs. Malicious users cannot launch a double-spending attack to unlock an output requiring $R_{i,j}$'s signature without $R_{i,j}$'s participation. We add a signature condition to the input of tx^{ep2} , requiring the signatures of all channel receivers in the SG corresponding to tx^{ep2} to unlock it. Consequently, if there is at least one honest key party, malicious users are unable to launch a double spending attack on tx^{relay} , tx^{ep2} , and tx^{p2} , which are required for other honest channel receivers to enforce payment on-chain.

The detailed definition of functions for generating transactions is shown in Fig. 6. We provide a comprehensive security analysis of the EPTT with the aforementioned strategies in Section V, demonstrating that our solution safeguards the balance security for honest intermediate parties, even in scenarios involving malicious key parties. For existing attacks (i.e., wormhole attacks and channel closure attacks), our protocol inherently resists wormhole attacks due to the atomicity in SAMCU being guaranteed by UTXO mechanism instead of secret passing. Additionally, our protocol defends against channel closure attacks as it prevents double spending, which is the core operation of such attacks. However, malicious key parties may launch a wormhole-like attack to steal transaction fees from normal parties rather than their funds. This type of attack causes relatively minor losses compared to traditional wormhole attacks. We discuss this attack in Appendix C.

4) *Other Properties*: In addition to the aforementioned security and privacy goals, our solution achieves performance improvements in the following aspects.

a) *Small size & low fee of on-chain transaction*: Compared with existing protocols, the total size of transactions for enforcing payment per person is reduced because we split the large enable payment transaction into several smaller ones. A channel receiver who incurs a dispute only needs to publish one small enable payment transaction to enforce her payment. For example, as shown in Fig. 4(b), in existing approaches, a channel receiver needs to publish tx^{ep} with 1 input and 15 outputs. In contrast, in our approach, the channel receiver in SG_4 only needs to generate a tx^{ep1} with 1 input, and 4 outputs for triggering other parties in UG, as shown in Fig. 5. After observing a tx^{ep1} on the blockchain, parties in other SGs only need to publish tx^{relay} and tx^{ep2} , where the total size of these two transactions is also smaller than the tx^{ep} in the existing approaches.

However, there remains a challenge in reducing the on-chain transaction fees for enforcing payments per person: If we employ the regular transaction fee payment mechanism (adding the fee to one of the inputs) in SAMCU, then every channel receiver needs to cover the fee for the transactions in her EPTT (except $\text{tx}^{\text{p1/p2}}$) because the funds in inputs of these transactions all originate from tx^{in} . Therefore, a receiver must pay not only for transactions for enforcing her payment (i.e., tx^{in} , tx^{ep1}) but also for transactions (i.e., tx^{relay} , tx^{ep2}) for enforcing other receivers' payment. To solve this problem, we employ Child-Pays-For-Parent [16] to pay the fees instead of the regular method. We can modify the transactions in EPTT to support CPFPP by increasing the value of the input by 1ϵ and adding an output with 1ϵ and no locking script in transactions tx^{relay} , $\text{tx}^{\text{ep1/ep2}}$. Although these transactions are set with 0 transaction fees, anyone can use the CPFPP strategy to pay the fees without changing the content of the parent transactions. Details of CPFPP are introduced in Appendix D. With CPFPP, parties only need to pay for the transaction enforcing their payments, thereby reducing the on-chain transaction fee per person.

b) *Few communication connections*: In existing approaches, each intermediary needs to establish network connections with all parties involved when participating in an MCU. In contrast, our solution requires intermediaries to establish connections only with the dealer, other parties within the same SG, and all key parties at most. This selective connection strategy reduces the network resources consumed by parties participating in MCUs.

IV. CONSTRUCTIONS

A. Building Blocks

1) *Digital Signature Scheme*: A digital signature scheme Σ is composed of three algorithms $\Sigma = (\text{Gen}, \text{Sign}, \text{Verify})$:

- $\text{sk}, \text{pk} \leftarrow \text{Gen}(1^\lambda)$ is a PPT algorithm that takes the security parameter 1^λ and outputs a public key pk and the corresponding secret key sk .
- $\sigma \leftarrow \text{Sign}(\text{sk}, m)$ is a PPT algorithm that takes a secret key sk and a message m as input and outputs a signature σ .

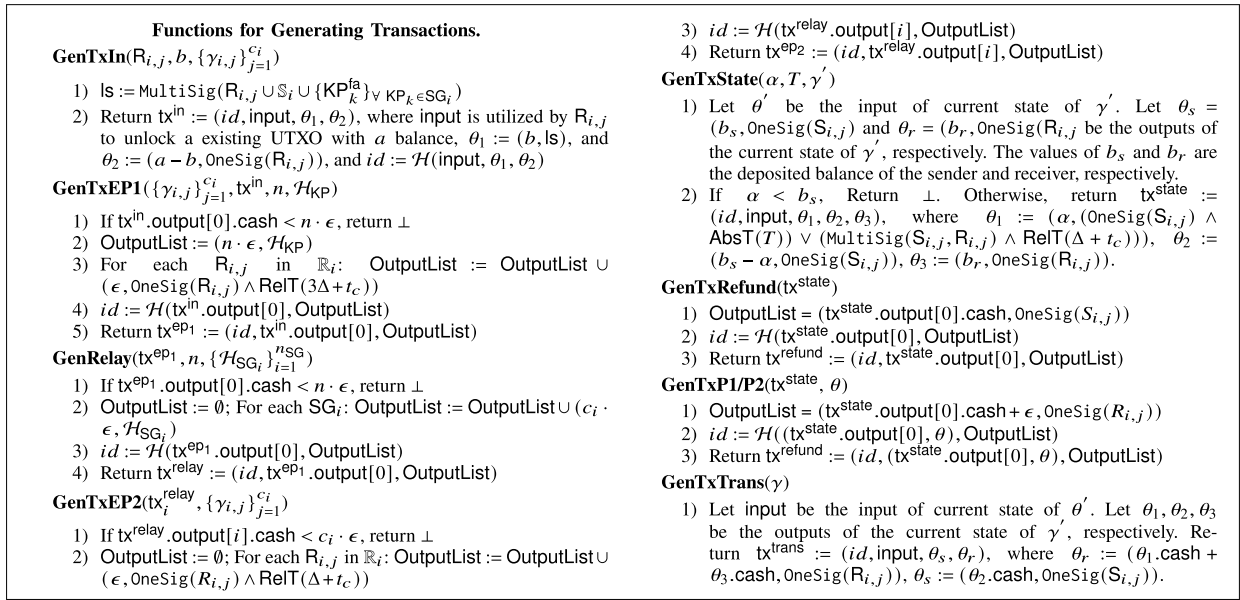


Fig. 6. Functions for generating transactions.

- $\{0, 1\} \leftarrow \text{Verify}(\sigma, m, \text{pk})$ is a DPT algorithm that takes signature σ , a message m , and a public key pk as input. It determines the validity of the signature by checking if σ was created by the secret key corresponding to pk for the given message m . If the signature is valid, the algorithm outputs 1; otherwise, it outputs 0.
- 2) *Payment Channel Networks*: A PCN is equipped with three following operations:
 - **createChannel**($\text{tx}^{\text{deposit}} \rightarrow \gamma$) Any two users can submit a transaction $\text{tx}^{\text{deposit}}$ to the blockchain to open a channel. This transaction has two inputs, each from one of the users. The output of this transaction is a 2-of-2 multi-signature address controlled by both parties. Additionally, the two parties will negotiate tx^{state} , which can be used to redeem their respective funds from the 2-of-2 multi-signature address. It is important to note that tx^{state} is not immediately recorded on the blockchain at this stage. Finally, this operation generates a payment channel γ with an identifier as the 2-of-2 multi-signature address.
 - **updateChannel**($\gamma, \text{tx}^{\text{state}}, t_u \rightarrow \{0, 1\}$) Either user can update the state of channel γ to the state defined by tx^{state} , where tx^{state} is the unsigned version of tx^{state} . When both users complete the negotiation and sign tx^{state} , this operation returns 1. Otherwise, it returns 0. The maximum execution time for this operation is t_u .
 - **closeChannel**(γ, t_c), Either user can close the channel by publishing the latest tx^{state} stored in γ , which represents the latest state of the channel, to the blockchain. t_c represents the maximum execution time for this operation.

B. Protocol Description

The formal construction of our protocol is described below and shown in Fig. 7.

1) *Initialization*: One of the payment senders is chosen as the dealer. After dealer splitting the updating graph

$\text{UG} := ((\gamma_i, \alpha_i)_{i=1}^n)$, she requires all the key parties to create the fresh addresses $\{KP_i^{\text{fa}}\}_{i=1}^{n_{\text{KP}}}$ for $\{KP_i\}_{i=1}^{n_{\text{KP}}}$ exclusively for this specific update. Based on $\{KP_i^{\text{fa}}\}_{i=1}^{n_{\text{KP}}}$, dealer generates the input and output parameters required for tx^{relay} . Subsequently, the dealer distributes the relevant parameters to all parties. If all parties agree to participate in this update by sending confirmation messages to dealer, the process will proceed to the next phase.

2) *Pre-Setup-SG*: For each SG, for each receiver, $R_{i,j}$ generate tx^{in} . The value of the output is $(c_i + n) \cdot \epsilon$. Subsequently, $R_{i,j}$ generate tx^{ep1} , with the last output being $\text{MultiSig}(\{KP_i^{\text{fa}}\}_{i=1}^{n_{\text{KP}}})$ and a value of $n \cdot \epsilon$. The remaining outputs of tx^{ep1} are allocated to send ϵ to other receivers within the same sub-graph. Finally, a two-step notification process ensures that each party is aware of the completion of generation by other parties before proceeding to the next phase.

3) *Pre-Setup-UG*: For each key party $KP_k \in \{KP_i\}_{i=1}^{n_{\text{KP}}}$, she first generates the corresponding tx^{relay} for all the received tx^{ep1} from the parties in her sub-graphs and sends tx^{relay} to all the other KPs. Then, once a key party receives tx^{relay} from all KPs, she generates the corresponding tx^{ep2} for each received tx^{relay} . Finally, KP_k sends all the generated tx^{relay} and tx^{ep2} to all parties in her sub-graphs. Upon receiving them, parties will check the correctness of the received transactions.

4) *Setup*: The channel senders create tx^{state} and $\text{tx}^{\text{refund}}$, along with $\text{tx}^{\text{P1/P2}}$ for each received $\text{tx}^{\text{ep1/ep2}}$. Then, they send tx^{state} , all $\text{tx}^{\text{P1/P2}}$ and the signatures for all $\text{tx}^{\text{P1/P2}}$ to their channel receivers, respectively. This step ensures that the receivers can post $\text{tx}^{\text{P1/P2}}$ on the blockchain to redeem their balances, regardless of which $\text{tx}^{\text{ep1/ep2}}$ that is eventually published. Subsequently, a two-step notification process ensures that each party knows the completion of checking received transactions by all other channel receivers before proceeding to the next phase.

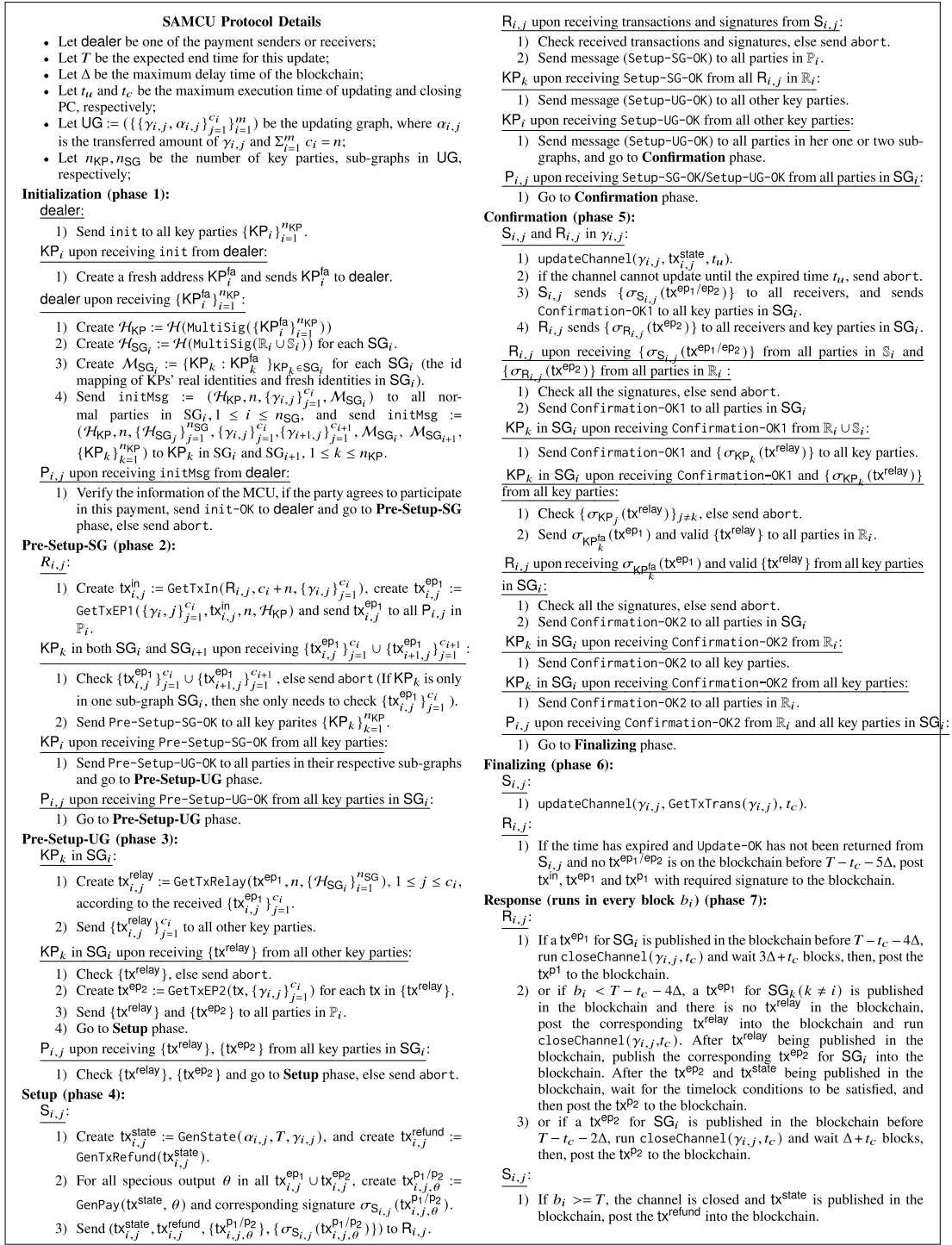


Fig. 7. SAMCU protocol.

5) *Confirmation*: After completing the two-step notification process in the last phase, each channel sender proceeds to update the channel with the channel receiver from its un-updated state to the transmit state (represented by tx^{state}) because she can ensure that all receivers possess $\text{tx}^{\text{p1/p2}}$ with signatures that allow them to claim the funds from tx^{state} . If the channel is successfully updated, the channel sender signs each $\text{tx}^{\text{ep1/ep2}}$ and sends it to the corresponding receiver, while the channel receiver signs each tx^{ep2} and sends it to

the corresponding receiver. When a channel receiver gets the signatures from all channel senders and receivers, she can meet the signature condition $\text{MultiSig}(\mathbb{S}_i)$ of her tx^{ep1} and meet all the signature conditions of her tx^{ep2} , so she sends **Confirmation-OK1** to all parties within the same sub-graph. When a key party gets message **Confirmation-OK1** from all parties in her one or two sub-graphs, she will send **Confirmation-OK1**, the signatures of $\{\text{tx}^{\text{relay}}\}$, to all other key parties. After that, when a key party receives

Confirmation-OK1 from all key parties, it means that all channels from other sub-graphs are successfully updated. Then, the key party signs each tx^{ep_1} and sends it and valid $\{\text{tx}^{\text{relay}}\}$ with all signatures to the corresponding channel receiver in her sub-graphs. When a receiver gets the signatures from all key parties with the same sub-graph, she is able to meet the third condition of her tx^{ep_1} on the blockchain, i.e., $\text{MultiSig}(\{\text{KP}_k^{\text{fa}}\} \vee \text{KP}_k \in \text{SG}_i)$. Subsequently, a two-step confirmation process ensures that each party knows that all channel receivers have valid tx^{ep_1} before proceeding to the next phase.

6) *Finalization*: After the last phase, every channel sender can ensure that all channels in UG are ready to be updated and all channel receivers have valid tx^{ep_1} . Then, she can start updating her channel with the channel receiver off-chain. In case a PC fails to be updated, the channel receiver can post her own tx^{in} , tx^{ep_1} , and tx^{p_1} to enforce payment.

7) *Response*: All participants should watch the blockchain's status and respond accordingly when new blocks are created on the blockchain. Each channel receiver needs to watch which transactions have appeared on the blockchain. In the event that a *Trigger* transaction appears in the blockchain, she needs to post the corresponding *response* transactions to the blockchain to forcibly redeem her funds in the payment channel following the corresponding EPTT if there is a dispute in her channel. Each channel sender also needs to actively watch the blockchain. If her receiver has not redeemed her funds by posting $\text{tx}^{\text{p}_1/\text{p}_2}$ before T , the sender needs to post $\text{tx}^{\text{refund}}$ to redeem her funds after T .

V. SECURITY ANALYSIS

A. Balance Security

Theorem 1: No honest intermediate party within SAMCU loses her funds.

Proof: We first define four states for payment channels: (a) We define a channel as *unchanged* when its state remains unaffected by the current update, typically occurring when a participant sends a message *abort* before the confirmation phase. (b) We define a channel as *revoked* when the channel sender uses tx^{state} to update the channel's state during the confirmation phase and subsequently redeems her funds from the channel using a $\text{tx}^{\text{refund}}$ transaction after time T . (c) We define a channel as *compensated* when the channel sender uses tx^{state} to update the channel's state during the confirmation phase and the receiver redeems her funds from the channel using $\text{tx}^{\text{p}_1/\text{p}_2}$ before time T . (d) We define a channel as *successful* when the channel sender uses tx^{trans} to update the channel's state during the finalization phase. For an honest intermediary $P_{i,j}$, we consider her upstream channel (where she acts as the receiver) and downstream channel (where she acts as the sender) as γ_l and γ_r , respectively. There are three scenarios that can compromise the balance security of $P_{i,j}$:

- γ_l is **unchanged**, γ_r is **successful or compensated**. When the upstream channel γ_l remains unchanged, the rational channel receiver of γ_l will not sign any $\text{tx}^{\text{ep}_1/\text{ep}_2}$, thus γ_r cannot be compensated. As the channel sender of γ_r ,

she also will not honestly update the state of γ_r off-chain. This leads to a contradiction.

- γ_l is **revoked**, γ_r is **successful**. A rational $P_{i,j}$ will not honestly update γ_r off-chain when she lacks the ability to enforce payment from γ_l . This leads to a contradiction.
- γ_l is **revoked**, γ_r is **compensated**. γ_l being revoked implies that $P_{i,j}$ cannot enforce the payment independently (i.e., she does not possess a valid transaction tx^{ep_1}). γ_r being compensated indicates that the channel receiver of γ_r has already published a valid $\text{tx}^{\text{ep}_1/\text{ep}_2}$. The validity of this transaction requires $P_{i,j}$'s signature, implying that $P_{i,j}$ is aware of the transaction and its children transactions. Then, if $P_{i,j}$ is a normal party, she can initiate her own $\text{tx}^{\text{p}_1/\text{p}_2}$ corresponding to $\text{tx}^{\text{ep}_1/\text{ep}_2}$ on the blockchain to redeem funds from γ_l . Otherwise, if $P_{i,j}$ is a key party, two scenarios exist: (a) γ_r was compensated by tx^{ep_1} and tx^{p_1} . In this situation, $P_{i,j}$ can initiate the corresponding tx^{relay} , tx^{ep_2} , and tx^{p_2} on the blockchain to redeem funds from γ_l . These transactions must be valid since the key party only signs tx^{ep_1} if its child transactions are valid according to the procedure of the protocol. (b) γ_r was compensated by tx^{ep_2} and tx^{p_2} . In this situation, the parent transaction tx^{relay} of tx^{ep_2} must also be on the blockchain. Then, $P_{i,j}$ can initiate tx^{ep_2} for their left SG and tx^{p_2} following the already-on-chained tx^{relay} on the blockchain to redeem funds from γ_l . Hence, this creates a contradiction as γ_l cannot be revoked.

All possible scenarios that compromise the balance security of an honest intermediary are contradictory. Therefore, we infer that no honest intermediary loses her funds within SAMCU. ■

B. Internal Anonymity

Theorem 2: SAMCU achieves internal anonymity when it runs in the honest case.

Sketch Proof: When the protocol runs without disputes, each key party only knows the public addresses of parties located in her SG(s), as well as the fresh addresses of other KPs. Similarly, every normal party only knows the public addresses of other parties within her SG and the fresh addresses of all KPs. However, as long as the protocol is executed honestly, the fresh addresses are not included on the blockchain, eliminating concerns about the possibility of linking fresh addresses to the public addresses of KPs. In summary, each party is only aware of the public addresses belonging to the parties in its neighborhood. Thus, we conclude that the SAMCU protocol achieves internal anonymity when the protocol runs without disputes. ■

VI. EVALUATION

A. Theoretical Analysis

We conduct a theoretical analysis between SAMCU and other comparisons (as shown in Table II). In this table, n represents the number of participating payment channels in all protocols. m and k represent the number of sub-graphs and the number of key parties in SAMCU. We assume that the n

channels can be evenly distributed among the m sub-graphs in SAMCU.

Compared to solutions for MHPs (i.e., LN, PT, Blitz), SAMCU achieves higher expressiveness (i.e., MCUs). In addition, SAMCU achieves constant setup time (i.e., generating all transactions within a finite number of rounds) and constant collateral time (i.e., parties can withdraw funds locked in transactions within constant time). On the other hand, to achieve constant setup and collateral times, like other solutions for MCUs (i.e., Thora), SAMCU incurs higher communication and computational costs compared to MHPs solutions to ensure secure synchronization within constant running time. Both communication and computation expenses in SAMCU increase complexity by one degree compared to solutions for MHPs. Furthermore, SAMCU employs transactions of size $\mathcal{O}(\frac{n}{m} + m)$ instead of $\mathcal{O}(1)$ for enforcing payment.

However, compared to the SOTA for MCUs (i.e., Thora), SAMCU offers the following improvements: (a) **Number of Connections**: Thora requires all parties to establish connections with every other party, resulting $\mathcal{O}(n^2)$ connections. In our approach, during the initialization phase, the `dealer` establishes n connections with all parties involved in SAMCU. As the progresses from the pre-setup to the confirmation phase, all parties need pairwise connections within each SG, totaling $\mathcal{O}(\frac{n^2}{m})$ connections. Communication between groups requires pairwise connections between key parties, totaling $\mathcal{O}(k^2)$ connections. Thus, the overall number of connections is $\mathcal{O}(\frac{n^2}{m} + k^2)$; (b) **On-Chain Transaction Size and Fee**: In protocols for MCUs, parties collaboratively create transactions to ensure their balance security, with the sizes of these transactions outlined in Table III. When disputes arise in channels, in Thora, disputing users must emit tx^{in} , tx^{ep} , and tx^{p} to the blockchain, totaling $988 + 36n$ bytes, with a complexity of $\mathcal{O}(n)$. In SAMCU, the receiver of the first disputed channel needs to emit tx^{in} , tx^{ep_1} , and tx^{p_1} to the blockchain, while parties in other SGs need to emit tx^{relay} , tx^{ep_2} , and tx^{p_2} to the blockchain. The total size of transactions for enforcing payment in SAMCU are $939 + 36\frac{n}{m}$ and $842 + 36(\frac{n}{m} + m)$ bytes, respectively, with a complexity of $\mathcal{O}(\frac{n}{m} + m)$.

Our approach does not optimize computational complexity or the data size of communication. Although the size of individual transactions is reduced, the overall size of the EPTTs remains comparable to Thora, still at $\mathcal{O}(n^2)$. Since most of the computational time is spent generating transactions and most of the communication data consists of these transactions, both the computational time complexity and the complexity of communication data size per person are consistent with the complexity of total transaction size, remaining at $\mathcal{O}(n^2)$, the same as Thora.

B. Experimental Results

We first implement a prototype of SAMCU in Python 3. We conduct a series of experiments aimed at (a) evaluating the size and fee of on-chain transactions, (b) evaluating the computation time, and (c) evaluating the communication connections, using a machine with Quad-Core Intel Core i5 (1.4 GHz).

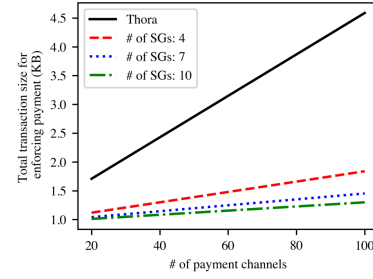


Fig. 8. Comparing total transaction sizes for enforcing payment between SAMCU and Thora.

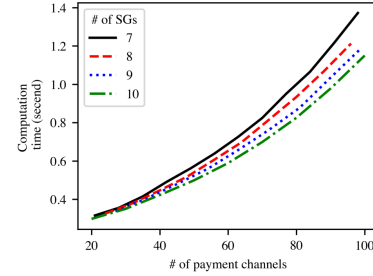


Fig. 9. Computation time for varying numbers of channels.

1) *On-chain Transaction Size and Fee*: We vary the number of PCs from 20 to 100 and illustrate the total transaction size for a disputing user to enforce payment in the different numbers of SGs in Fig. 8. The result shows the total transaction size for a disputing user to enforce payment rises nearly linearly in all cases as the number of PCs increases. For the same number of PCs, the total transaction size for a user to enforce payment decreases as the number of SGs increases. In the current Bitcoin environment, recording 500 bytes of data requires a transaction fee of 1 USD. When n is 100 and m is 10, the on-chain transaction fees significantly decrease from 9.2 USD to 3.1 USD, reducing the cost of enforcing payments compared to Thora.

2) *Computation Time*: We vary the number of PCs from 20 to 100 and illustrate the computational time for a key party with different numbers of SGs in Fig. 9. The results show that the computational time for a key party exhibits quadratic growth in all cases as the number of PCs increases. For the same number of PCs, the computational time for a key party decreases as the number of SGs increases. When updating 100 channels, the total computational time for a key party is less than 1.5s.

3) *Communication Connections*: We vary the number of PCs from 20 to 100 and illustrate the number of communication connections in the different numbers of SGs in Fig. 10. In all cases, there are 3 chains in UG. The result shows that the total number of communication connections exhibits quadratic growth in all cases as the number of PCs increases. For the same number of PCs, the total number of communication connections decreases as the number of SGs increases. When n is 100, m is 10, the connection count decreases from 5.35K (in Thora) to 1.57K (in SAMCU), signifying a reduction of 70%. This enhancement renders the protocol more practical for large-scale MCUs.

TABLE II
THEORETICAL COMPARISONS BETWEEN SAMCU AND OTHER RELATED PROTOCOLS

Scheme	Time Complexity		Computational Complexity		Communication Complexity						Transaction Fee&Size
	Setup Time	Compensation Time	Transactions Generation	Signatures Generation	Connections			Data Size			
					Dealer	Party	Overall	Dealer	Party	Overall	
LN	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
AMCU	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$
PT	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$
Blitz	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$
Thora	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n)$
SAMCU	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	$\mathcal{O}(\frac{n}{m} + k)$	$\mathcal{O}(\frac{n^2}{m} + k^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^3)$	$\mathcal{O}(\frac{n}{m} + m)$

TABLE III
COMPARING TRANSACTION SIZES (BYTE)

Methods	tx^n	$tx^{ep/ep1}$	tx^{relay}	tx^{ep2}	$tx^{P/P1/P2}$	tx^{state}	tx^{refund}
Thora	223	$256 + 36n$	null	null	500	368	272
SAMCU	223	$162 + 36(n/m + 1)$	$162 + 36m$	$162 + 36(n/m)$	500	368	272

* $tx^{ep/ep1/ep2}$ and tx^{relay} require an extra 9 bytes to support CPFP.

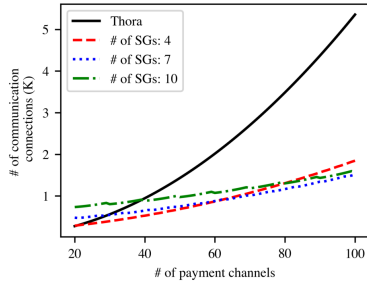


Fig. 10. Comparing the number of communication connections between SAMCU and Thora.

VII. RELATED WORK

Anonymization techniques have been widely used to protect privacy in various domains (e.g., distributed learning [17], data sharing [18], [19], and person re-identification [20]). Although anonymity is a key feature of blockchain technology, external observers can still correlate different transactions made by the same user through her account addresses, ultimately tracing user behavior to de-anonymize the user entirely [21]. In recent years, significant efforts have been devoted to enhancing identity privacy on blockchains. In addition to off-chain PCNs for achieving anonymity, anonymous cryptocurrencies (e.g., Monero [22] and Zcash [23]) use advanced cryptographic techniques, such as Linkable Ring Signatures [22] and zk-SNARKs [23], to provide greater transaction anonymity than Bitcoin.

Another technique for achieving anonymity is through third-party mixing services. In this approach, Senders send their coins to a mixing pool, and the service redistributes the coins among receivers to obscure the relationship between senders and receivers. CoinJoin [24] is one of the first methods to provide a mixing service for Bitcoin, where multiple users combine their Bitcoin transactions into a single transaction. To eliminate the need for trusted users in CoinJoin, CoinShuffle [25] employs an anonymous group communication protocol to conceal the participants' identities from each other.

VIII. CONCLUSION

In this paper, we propose SAMCU, a secure and anonymous multi-channel update protocol for PCN. Unlike traditional solutions that sacrifice privacy for multi-channel

updates, SAMCU achieves both multi-channel updates and sender/receiver privacy. Our SAMCU has lower on-chain transaction fees per person and fewer communication connections compared to previous work. For instance, when simultaneously updating 100 channels, our approach yields around 70% savings in communication connections and reduces on-chain transaction fees by about 66% relative to the state-of-the-art. Our SAMCU makes multi-channel updates more practical, particularly in large-scale multi-channels. In future work, we intend to enhance the interoperability of SAMCU, enabling secure and privacy-preserving multi-channel updates across different blockchains.

REFERENCES

- [1] G. Malavolta, P. Moreno-Sanchez, A. Kate, M. Maffei, and S. Ravi, "Concurrency and privacy with payment-channel networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 455–471.
- [2] C. Egger, P. Moreno-Sanchez, and M. Maffei, "Atomic multi-channel updates with constant collateral in Bitcoin-compatible payment-channel networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 801–815.
- [3] M. Jounen, M. Larangeira, and K. Tanaka, "Payment trees: Low collateral payments for payment channel networks," in *Financial Cryptography and Data Security*. Berlin, Germany: Springer, 2021, pp. 189–208.
- [4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, "Blitz: Secure multi-hop payments without two-phase commits," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 4043–4060.
- [5] L. Aumayr, K. Abbaszadeh, and M. Maffei, "Thora: Atomic and privacy-preserving multi-channel updates," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 165–178.
- [6] (2012). *BIP 0016*. [Online]. Available: https://en.bitcoin.it/wiki/BIP_0016
- [7] *Eclair Network*. Accessed: Feb. 1, 2024. [Online]. Available: <https://github.com/ACINQ/eclair>
- [8] G. Malavolta, P. Moreno-Sanchez, C. Schneidewind, A. Kate, and M. Maffei, "Anonymous multi-hop locks for blockchain scalability and interoperability," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2019, pp. 1–30.
- [9] X. Wang, C. Lin, X. Huang, and D. He, "Anonymity-enhancing multi-hop locks for monero-enabled payment channel networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2438–2453, 2024.
- [10] A. Miller, I. Bentov, S. Bakshi, R. Kumaresan, and P. McCorry, "Sprites and state channels: Payment networks that go faster than lightning," in *Proc. Int. Conf. Financ. Cryptogr. Data Secur.* Berlin, Germany: Springer, 2019, pp. 508–526.
- [11] G. Malavolta, P. Moreno-Sanchez, A. Kate, and M. Maffei, "SilentWhispers: Enforcing security and privacy in decentralized credit networks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2017, pp. 1–15.
- [12] S. Roos, P. Moreno-Sanchez, A. Kate, and I. Goldberg, "Settling payments fast and private: Efficient decentralized routing for path-based transactions," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [13] P. Wang, H. Xu, X. Jin, and T. Wang, "Flash: Efficient dynamic routing for offchain networks," in *Proc. 15th Int. Conf. Emerg. Netw. Exp. Technol.*, Dec. 2019, pp. 370–381.
- [14] V. Sivaraman et al., "High throughput cryptocurrency routing in payment channel networks," in *Proc. USENIX Symp. Netw. Syst. Des. Implement.*, 2020, pp. 777–796.

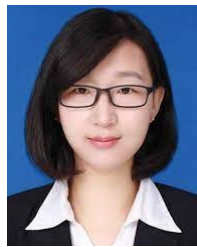
- [15] *Bitcoin Visuals*. Accessed: Feb. 1, 2024. [Online]. Available: <https://bitcoinvisuals.com/ln-eccentricity/>
- [16] *Child Pays For Parent*. Accessed: Feb. 1, 2024. [Online]. Available: <https://bitcoincore.org/en/releases/0.13.0/#mining-transaction-selection-child-pays-for-parent>
- [17] Y. Jiang, K. Zhang, Y. Qian, and L. Zhou, "Anonymous and efficient authentication scheme for privacy-preserving distributed learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2227–2240, 2022.
- [18] J. Sun, G. Xu, T. Zhang, X. Yang, M. Alazab, and R. H. Deng, "Privacy-aware and security-enhanced efficient matchmaking encryption," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 4345–4360, 2023.
- [19] J. Sun, G. Xu, T. Zhang, X. Yang, M. Alazab, and R. H. Deng, "Verifiable, fair and privacy-preserving broadcast authorization for flexible data sharing in clouds," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 683–698, 2023.
- [20] M. Ye, W. Shen, J. Zhang, Y. Yang, and B. Du, "SecureReID: Privacy-preserving anonymization for person re-identification," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2840–2853, 2024.
- [21] H. Yousaf, G. Kappos, and S. Meiklejohn, "Tracing transactions across cryptocurrency ledgers," in *Proc. USENIX Secur. Symp.*, 2019, pp. 837–850.
- [22] N. Van Saberhagen. (2013). *Cryptonote V 2.0*. [Online]. Available: <https://api.semanticscholar.org/CorpusID>
- [23] (2016). *Zcash*. [Online]. Available: <https://z.cash/>
- [24] G. Maxwell. (Dec. 2013). *Coinjoin: Bitcoin Privacy for the Real World*. [Online]. Available: <https://bitcointalk.org/?topic=279249>
- [25] T. Ruffing, P. Moreno-Sanchez, and A. Kate, "CoinShuffle: Practical decentralized coin mixing for Bitcoin," in *Proc. 19th Eur. Symp. Res. Comput. Secur.-(ESORICS)*, 2014, pp. 345–364.
- [26] A. Mizrahi and A. Zohar, "Congestion attacks in payment channel networks," in *Proc. Int. Conf. Financ. Cryptogr. Data Security (FC)*. Berlin, Germany: Springer, 2021, pp. 170–188.



Jianhuan Wang received the B.Eng. degree from Jinan University, China, and the M.S. degree from the Department of Computing, The Hong Kong Polytechnic University, where he is currently pursuing the Ph.D. degree under the supervision of Dr. Bin Xiao. His research interests include blockchain security and DID security.



Shang Gao (Member, IEEE) received the B.S. degree from Hangzhou Dianzi University, China, in 2010, the M.E. degree from Southeast University, China, in 2014, and the Ph.D. degree from The Hong Kong Polytechnic University, Hong Kong, in 2019. After graduation, he was with Microsoft China for one year. He is currently a Research Assistant Professor with the Department of Computing, The Hong Kong Polytechnic University. He specifically specializes in applied cryptography and zero-knowledge proofs, with a keen interest in areas, such as RingCT protocols, zkSNARKs, Layer2 security, and recursive SNARKs. His work has been published in many top-tier conferences and journals, including IEEE S&P, ACM CCS, IEEE INFOCOM, IEEE/ACM TRANSACTIONS ON NETWORKING, and IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING. His primary research interests include computer security and information security.



Guyue Li (Member, IEEE) received the B.S. degree in information science and technology and the Ph.D. degree in information security from Southeast University, Nanjing, China, in 2011 and 2017, respectively. From June 2014 to August 2014, she was a Visiting Student with the Department of Electrical Engineering, Tampere University of Technology, Tampere, Finland. She is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University, and a Visiting Scholar with Tampere University of Technology and Université Gustave Eiffel (ESIEE PARIS), Noisy-le-Grand, France. Her current research interests include wireless network attacks, physical-layer security solutions for 5G and 6G, secret key generation, radio frequency fingerprints, and reconfigurable intelligent surfaces. She was a recipient of the Young Scientist Awarded by the International Union of Radio Science (URSI), the Youth Science and Technology Prize of Jiangsu Cyber Security Association, and the A-Level Zhishan Scholar of Southeast University. She was the Workshop Co-Chair of the IEEE Conference on Vehicular Technology (VTC) from 2021 to 2022. She is currently serving as an Editor for IEEE COMMUNICATION LETTERS and an Associate Editor for EURASIP Journal on Wireless Communications and Networking.



Keke Gai (Senior Member, IEEE) received the B.Eng. degree in automation from Nanjing University of Science and Technology, Nanjing, China, in 2004, the M.B.A. degree in business administration from Lawrence Technological University, Southfield, MI, USA, in 2009, the M.E.T. degree in educational technology from The University of British Columbia, Vancouver, BC, Canada, in 2010, the M.S. degree in information technology from Lawrence Technological University in 2014, and the Ph.D. degree in computer science from Pace University, New York, NY, USA. He is currently an Associate Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China. He has published three books and more than 130 peer-reviewed journal articles/conference papers, including eight ESI Highly Cited Papers in 2020. His research interests include cyber security, blockchain, edge computing, cloud computing, and reinforcement learning. He received five IEEE Best Paper Awards, such as TrustCom 2018 and HPCC 2018, and two IEEE best student paper awards in recent five years.



Bin Xiao (Fellow, IEEE) received the B.Sc. and M.Sc. degrees in electronics engineering from Fudan University, China, and the Ph.D. degree in computer science from The University of Texas at Dallas, Richardson, TX, USA. He is currently a Professor with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong. He has published more than 200 technical papers in international top journals and conferences. His research interests include AI and network security, data privacy, Web3, and blockchain systems. He is an Associate Editor of IEEE TRANSACTIONS ON CLOUD COMPUTING and IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING. He is the Chair of the IEEE ComSoc CISTC Committee from 2024 to 2025. He has been the Track Co-Chair of IEEE ICDCS2022, the Symposium Track Co-Chair of IEEE Globecom 2024, ICC2020, ICC 2018, and Globecom 2017, and the General Chair of IEEE SECON 2018. He has been an Associate Editor of the IEEE INTERNET OF THINGS JOURNAL and *Journal of Parallel and Distributed Computing* (Elsevier).